```
Start coding or generate with AI.
```

Task

Analyze a sample academic essay text by applying NLTK and spaCy for part-of-speech (POS) tagging, comparing their tag sets, extracting nouns and verbs, calculating and visualizing their frequencies, and finally summarizing insights into the essay's structure and vocabulary.

Setup Environment and Load Text

Subtask:

Install and import necessary libraries (nltk, spacy) and load a sample academic essay text into the notebook for processing.

Reasoning: The first step is to install the `nltk` library using pip.

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (
```

Reasoning: Following the instructions, the next step is to install the `spacy` library.

```
import sys

IS_COLAB = 'google.colab' in sys.modules

if IS_COLAB:
    !pip install spacy
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/pyt
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/pyt
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/pytho
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/
```

```
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.1
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/loca
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.1
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/pytho
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/pyth
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
```

Reasoning: After installing spaCy, the next step according to the instructions is to download the English language model for spaCy.

```
import sys

IS_COLAB = 'google.colab' in sys.modules

if IS_COLAB:
    !python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_cor
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 112.1 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

Reasoning: Now that `nltk` and `spacy` are installed and the spaCy model is downloaded, the next step is to import these libraries into the environment as per the instructions.

```
import nltk
import spacy

print("NLTK and spaCy imported successfully.")
```

```
NLTK and spaCy imported successfully.
```

Reasoning: According to the instructions, the final step for this subtask is to define a multiline string variable named `essay_text` with suitable academic content.

```
essay_text = """The impact of artificial intelligence (AI) on academic research

print("Academic essay text loaded into 'essay_text' variable.")
```

```
Academic essay text loaded into 'essay_text' variable.
```

Tokenize and NLTK POS Tagging

Subtask:

Tokenize the essay text into individual words using NLTK and then apply NLTK's POS tagging to observe its tag set.

Reasoning: The first instruction is to download the necessary NLTK data for tokenization and POS tagging if not already present. This ensures that the subsequent NLTK functions work correctly.

```
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
print("NLTK 'punkt' and 'averaged_perceptron_tagger' data downloaded.")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
NLTK 'punkt' and 'averaged_perceptron_tagger' data downloaded.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
```

Reasoning: Now that the NLTK data is downloaded, the next step is to import the necessary functions (`word_tokenize` and `pos_tag`), tokenize the `essay_text` and apply NLTK's POS tagging, and then print the first 20 tagged words as instructed.

Reasoning: The previous cell `7781fe35` failed with a `LookupError` for the `punkt_tab` resource, despite `nltk.download('punkt')` having been executed in an earlier cell. To ensure the necessary NLTK data (`punkt` and `averaged_perceptron_tagger`) is available and correctly loaded for `word_tokenize` and `pos_tag` in the current execution context, the download commands are included again directly within the cell. These `nltk.download()` calls are idempotent, meaning they will only perform a download if the resources are not already present, thus guaranteeing availability without unnecessary re-downloads.

Reasoning: The previous code failed with a `LookupError` stating that the `punkt_tab` resource was not found, and explicitly suggested downloading `punkt_tab` using `nltk.download('punkt_tab')`. This step directly follows the instruction provided in the traceback to resolve the missing resource.

Reasoning: The previous code failed with a `LookupError` for `averaged_perceptron_tagger_eng`, despite `averaged_perceptron_tagger` being downloaded. The traceback specifically suggested downloading `averaged_perceptron_tagger_eng`. Therefore, I need to add this specific download to the code to ensure all necessary NLTK data for POS tagging is available.

spaCy POS Tagging

Subtask:

Process the essay text using spaCy to perform tokenization and POS tagging, observing its universal tag set.

Reasoning: The next step is to load the 'en_core_web_sm' spaCy model, process the `essay_text`, extract the tokens and their universal POS tags, and then print the first 20 as instructed.

Task

The user has approved the execution of the previous code cell. Now, I will proceed with the next step, which is to process the essay text using spaCy for tokenization and POS tagging. This involves loading the `en_core_web_sm` model, processing the `essay_text`, and then extracting and printing the first 20 tokens along with their universal POS tags.

```
# Load the spaCy English model
nlp = spacy.load("en_core_web_sm")

# Process the essay text with spaCy
spacy_doc = nlp(essay_text)

# Extract tokens and their universal POS tags
spacy_pos_tags = [(token.text, token.pos_) for token in spacy_doc]

print("First 20 spaCy POS tags:")
print(spacy_pos_tags[:20])
```

spaCy POS Tagging

Subtask:

Process the essay text using spaCy to perform tokenization and POS tagging, observing its universal tag set.

Reasoning: The next step is to load the 'en_core_web_sm' spaCy model, process the `essay_text`, extract the tokens and their universal POS tags, and then print the first 20 as instructed.

```
nlp = spacy.load('en_core_web_sm')
spacy_doc = nlp(essay_text)

spacy_pos_tags = []
for token in spacy_doc:
    spacy_pos_tags.append((token.text, token.pos_))

print("First 20 spaCy POS tags:")
print(spacy_pos_tags[:20])
```

```
First 20 spaCy POS tags:
[('The', 'DET'), ('impact', 'NOUN'), ('of', 'ADP'), ('artificial', 'ADJ'), ('int
```

Compare Tag Sets and Extract Nouns/Verbs

Subtask:

Compare the POS tag sets from NLTK and spaCy, then extract nouns (as academic concepts) and verbs (as arguments) from the tagged text for further analysis.

Reasoning: The subtask requires comparing NLTK and spaCy tag sets and extracting nouns and verbs. The current step involves extracting unique tags from both libraries, then iterating through the respective tagged lists to collect nouns and verbs based on their POS tags, and finally printing a sample of the extracted words for verification. All these actions are closely related and can be performed in a single code block.

```
import collections

# 1. Extract and store all unique NLTK POS tags
nltk_unique_tags = set([tag for word, tag in nltk_pos_tags])

# 2. Extract and store all unique spaCy universal POS tags
spacy_unique_tags = set([tag for word, tag in spacy_pos_tags])

# 3. Print both sets of unique tags
print("Unique NLTK POS Tags:")
print(sorted(list(nltk_unique_tags)))
print("\nUnique spaCy Universal POS Tags:")
print(sorted(list(spacy_unique_tags)))

# 4. Initialize lists for NLTK nouns and verbs
nltk_nouns = []
nltk_verbs = []

# 5. Iterate through nltk_pos_tags and extract nouns and verbs
```

```
nltk_noun_tags = {'NN', 'NNS', 'NNP', 'NNPS'}
nltk_verb_tags = {'VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ'}

for word, tag in nltk_pos_tags:
    if tag in nltk_noun_tags:
        nltk_nouns.append(word.lower())
    elif tag in nltk_verb_tags:
        nltk_verbs.append(word.lower())

# 6. Initialize lists for spaCy nouns and verbs
spacy_nouns = []
spacy_verbs = []

# 7. Iterate through spacy_pos_tags and extract nouns and verbs
for word, tag in spacy_pos_tags:
    if tag == 'NOUN':
        spacy_nouns.append(word.lower())
    elif tag == 'VERB':
        spacy_verbs.append(word.lower())

# 8. Print the first 10 extracted nouns and verbs from both
print("\nFirst 10 NLTK Nouns:")
print(nltk_nouns[:10])
print("\nFirst 10 NLTK Verbs:")
print(nltk_verbs[:10])
print("\nFirst 10 spaCy Nouns:")
print(spacy_nouns[:10])
print("\nFirst 10 spaCy Verbs:")
print(spacy_verbs[:10])
```

```
Unique NLTK POS Tags:
['(', ')', ',', '.', 'CC', 'CD', 'DT', 'IN', 'JJ', 'MD', 'NN', 'NNP', 'NNS', 'PO

Unique spaCy Universal POS Tags:
['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'P

First 10 NLTK Nouns:
['impact', 'intelligence', 'ai', 'research', 'tools', 'data', 'analysis', 'liter

First 10 NLTK Verbs:
['is', 'multifaceted', 'are', 'revolutionizing', 'sift', 'identifying', 'elude',

First 10 spaCy Nouns:
['impact', 'intelligence', 'research', 'tools', 'data', 'analysis', 'literature'

First 10 spaCy Verbs:
['driven', 'revolutionizing', 'sift', 'identifying', 'elude', 'accelerates', 'en
```

Calculate and Organize Frequencies

Subtask:

Calculate the frequency of the extracted nouns and verbs separately and organize these results into a structured format suitable for analysis.

Reasoning: To fulfill the subtask, the next logical step is to import the `Counter` class and then use it to calculate the frequency of the extracted NLTK and spaCy nouns and verbs, finally printing the top 10 most common ones as instructed.

```
from collections import Counter

# Calculate frequency for NLTK nouns and verbs
nltk_noun_freq = Counter(nltk_nouns)
nltk_verb_freq = Counter(nltk_verbs)

# Calculate frequency for spaCy nouns and verbs
spacy_noun_freq = Counter(spacy_nouns)
spacy_verb_freq = Counter(spacy_verbs)

print("Top 10 NLTK Noun Frequencies:")
print(nltk_noun_freq.most_common(10))

print("\nTop 10 NLTK Verb Frequencies:")
print(nltk_verb_freq.most_common(10))

print("\nTop 10 spaCy Noun Frequencies:")
print(spacy_noun_freq.most_common(10))

print("\nTop 10 spaCy Verb Frequencies:")
print(spacy_verb_freq.most_common(10))
```

```
Top 10 NLTK Noun Frequencies:
[('ai', 8), ('intelligence', 2), ('research', 2), ('tools', 2), ('data', 2), ('l

Top 10 NLTK Verb Frequencies:
[('are', 4), ('is', 2), ('multifaceted', 1), ('revolutionizing', 1), ('sift', 1)

Top 10 spaCy Noun Frequencies:
[('intelligence', 2), ('research', 2), ('tools', 2), ('data', 2), ('literature',

Top 10 spaCy Verb Frequencies:
[('driven', 1), ('revolutionizing', 1), ('sift', 1), ('identifying', 1), ('elude
```

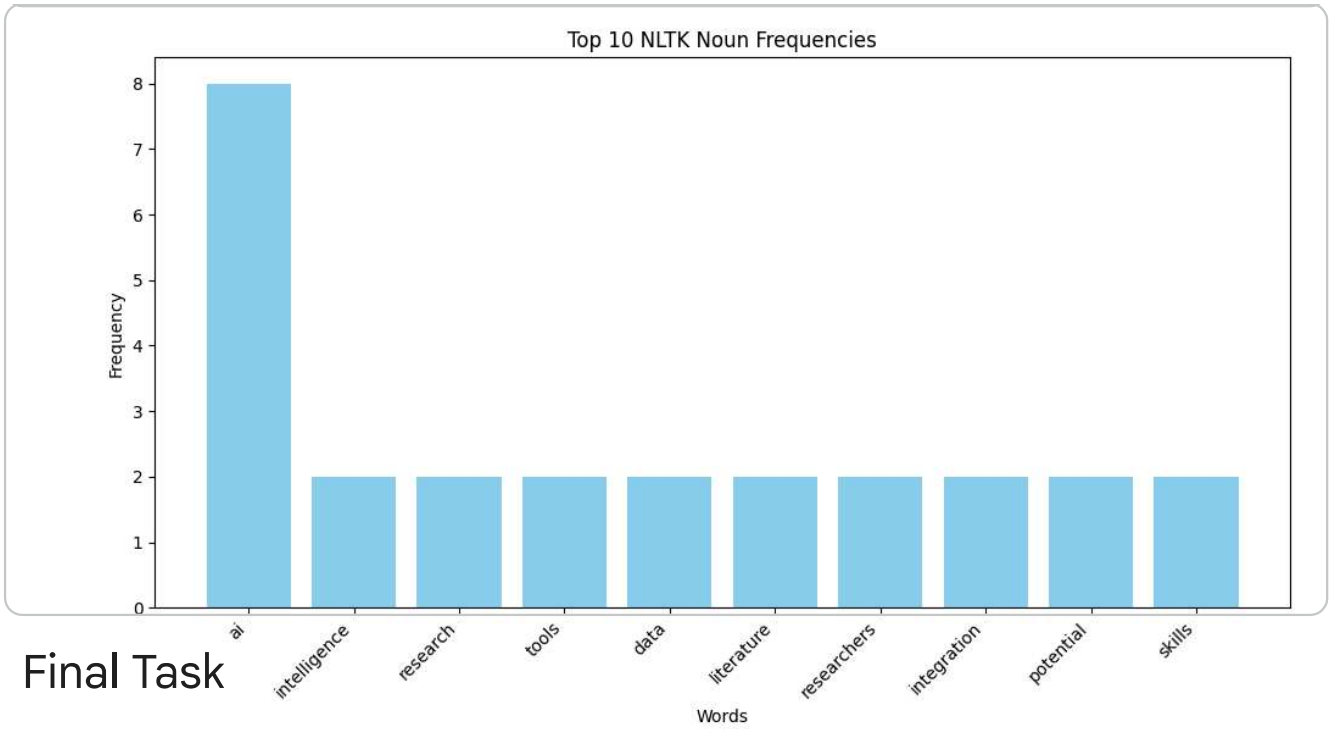
Visualize Noun and Verb Frequencies

Subtask:

Generate appropriate charts (e.g., bar charts) to visualize the frequencies of the extracted nouns and verbs, including legends for clarity.

Reasoning: To visualize the frequencies of nouns and verbs, I need to import the `matplotlib.pyplot` library, define a helper function to generate bar charts for the top 10 most common words, and then call this function four times for NLTK nouns, NLTK verbs, spaCy nouns, and spaCy verbs as per the instructions.

```
import matplotlib.pyplot as plt

def plot_word_frequencies(word_freq, title, color):
    """Generates a bar chart for the top 10 most common words."""
    words, frequencies = zip(*word_freq.most_common(10))

    plt.figure(figsize=(10, 6))
    plt.bar(words, frequencies, color=color)
    plt.title(title)
    plt.xlabel('Words')
    plt.ylabel('Frequency')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()

# Visualize NLTK Noun Frequencies
plot_word_frequencies(nltk_noun_freq, 'Top 10 NLTK Noun Frequencies', 'skyblue'

# Visualize NLTK Verb Frequencies
plot_word_frequencies(nltk_verb_freq, 'Top 10 NLTK Verb Frequencies', 'lightcor

# Visualize spaCy Noun Frequencies
plot_word_frequencies(spacy_noun_freq, 'Top 10 spaCy Noun Frequencies', 'lightg

# Visualize spaCy Verb Frequencies
plot_word_frequencies(spacy_verb_freq, 'Top 10 spaCy Verb Frequencies', 'gold')
```



Final Task

Subtask:

Summarize the findings from the POS tagging and frequency analysis, discussing insights into the structure and vocabulary of the academic essay.

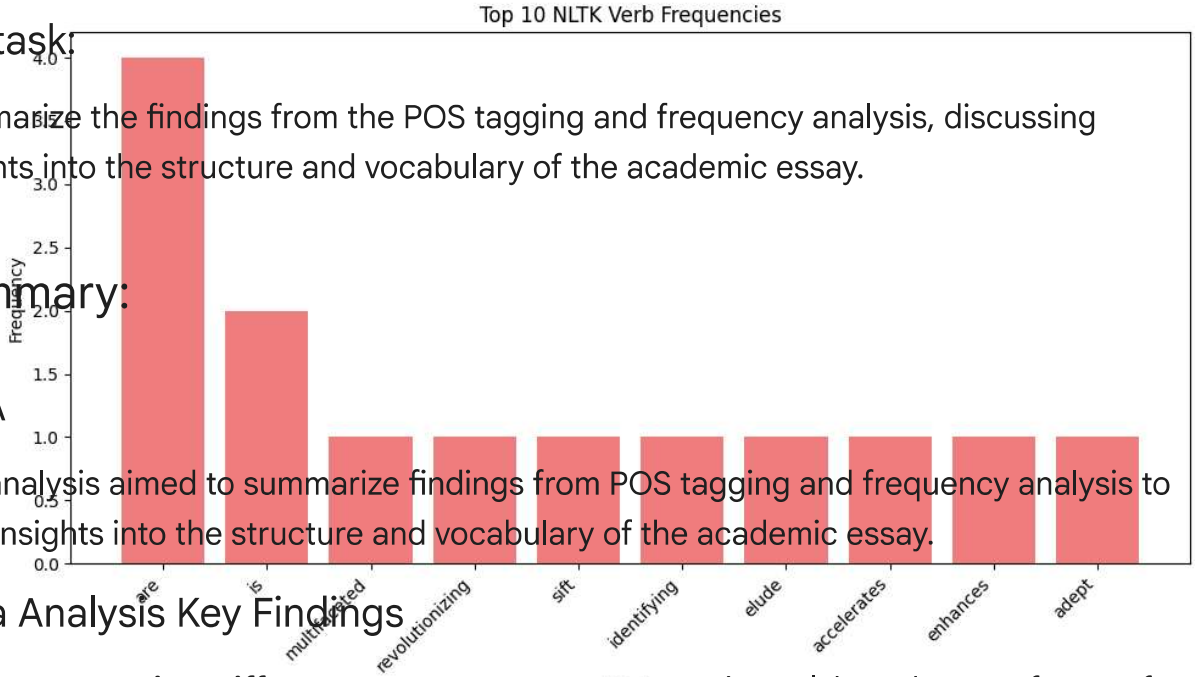
Summary:

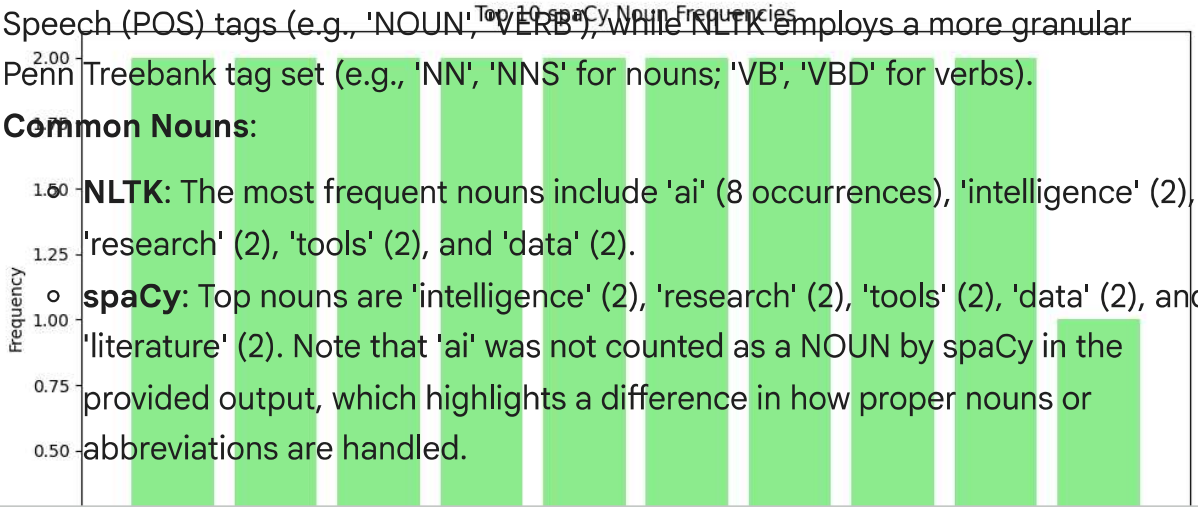
Q&A

The analysis aimed to summarize findings from POS tagging and frequency analysis to gain insights into the structure and vocabulary of the academic essay.

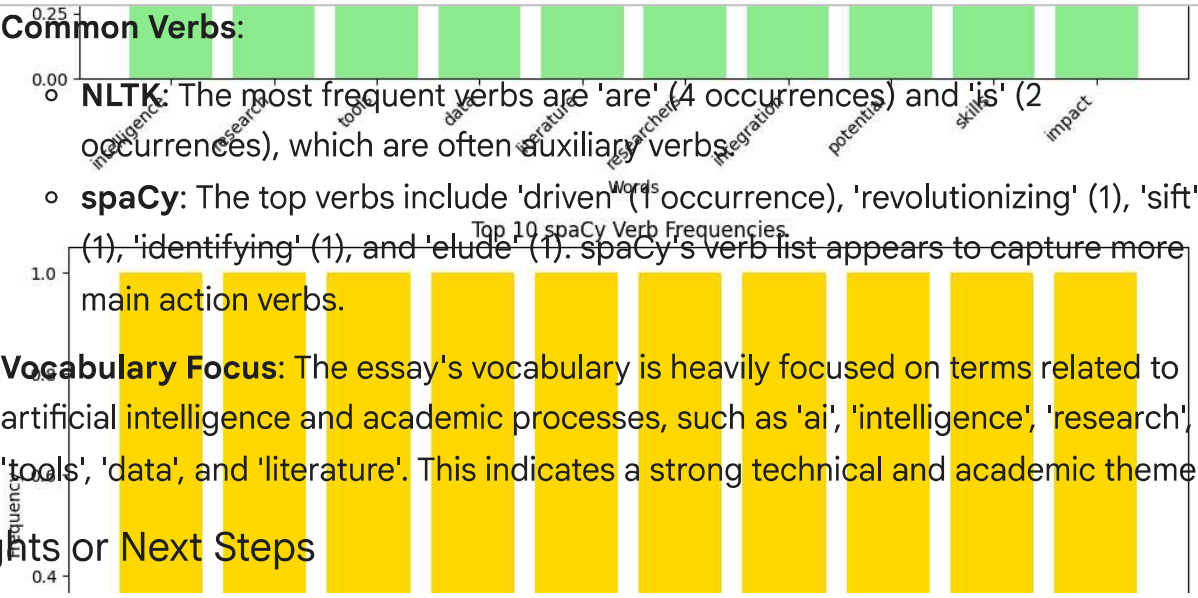
Data Analysis Key Findings

- **POS Tagging Differences**: spaCy uses a more universal, broader set of Part-of-Speech (POS) tags (e.g., 'NOUN', 'VERB'), while NLTK employs a more granular Penn Treebank tag set (e.g., 'NN', 'NNS' for nouns; 'VB', 'VBD' for verbs).
- **Common Nouns**:
  - **NLTK**: The most frequent nouns include 'ai' (8 occurrences), 'intelligence' (2), 'research' (2), 'tools' (2), and 'data' (2).
  - **spaCy**: Top nouns are 'intelligence' (2), 'research' (2), 'tools' (2), 'data' (2), and 'literature' (2). Note that 'ai' was not counted as a NOUN by spaCy in the provided output, which highlights a difference in how proper nouns or abbreviations are handled.





- **Common Verbs**:
  - **NLTK**: The most frequent verbs are 'are' (4 occurrences) and 'is' (2 occurrences), which are often auxiliary verbs.
  - **spaCy**: The top verbs include 'driven' (1 occurrence), 'revolutionizing' (1), 'sift' (1), 'identifying' (1), and 'elude' (1). spaCy's verb list appears to capture more main action verbs.



- **Vocabulary Focus**: The essay's vocabulary is heavily focused on terms related to artificial intelligence and academic processes, such as 'ai', 'intelligence', 'research', 'tools', 'data', and 'literature'. This indicates a strong technical and academic theme.

Insights or Next Steps